

Universidad Veracruzana

Instrumentación Electrónica

Sistema Modular de Nodos IoT
POO

Profesor:
Dominguez Chavez Alfonso

Alumno:
Hurtado Vasconcelos Rafael Alejandro
Romero Hernandez Luis Horacio
Landa Cervantes Juan Giovani
Vasquez Espinoza Santiago Maximiliano

1. Introducción

En esta práctica se desarrolló un sistema modular de nodos IoT utilizando el lenguaje de programación Python, aplicando los conceptos de herencia, encapsulamiento y polimorfismo de la programación orientada a objetos.
El proyecto se enfocó en simular un sistema de control de acceso mediante tecnología RFID (Identificación por Radiofrecuencia), en el cual se representa un lector de tarjetas y una cerradura electrónica.
Para organizar el sistema se implementó una clase principal denominada NodoIoT, que contiene los atributos y métodos comunes de los dispositivos conectados a una red. A partir de esta clase se desarrollaron dos clases hijas: NodoSensor, encargada de representar el lector RFID, y NodoActuador, que simula el funcionamiento de una cerradura electrónica.
Finalmente, se implementó un programa principal que permite verificar tarjetas autorizadas y mostrar el estado de los dispositivos mediante un bucle polimórfico, evitando el uso de condicionales para identificar el tipo de nodo.

2. Objetivos

Objetivo general

Desarrollar un sistema modular de control de acceso RFID utilizando Python, aplicando los principios de herencia, encapsulamiento y polimorfismo para simular el funcionamiento de dispositivos IoT.

Objetivos específicos

- Implementar una clase padre NodoIoT con atributos y métodos comunes.
- Crear clases hijas que representen un lector RFID y una cerradura electrónica.
- Utilizar super() para reutilizar el constructor de la clase padre.
- Aplicar encapsulamiento mediante atributos privados y métodos de validación.
- Sobrescribir el método enviar_telemetria() en las clases hijas.
- Implementar un bucle polimórfico para monitorear los dispositivos sin identificar su tipo mediante condicionales.
- Simular el acceso autorizado y no autorizado mediante códigos UID.

3. Desarrollo de la práctica

3.1 Diseño del sistema

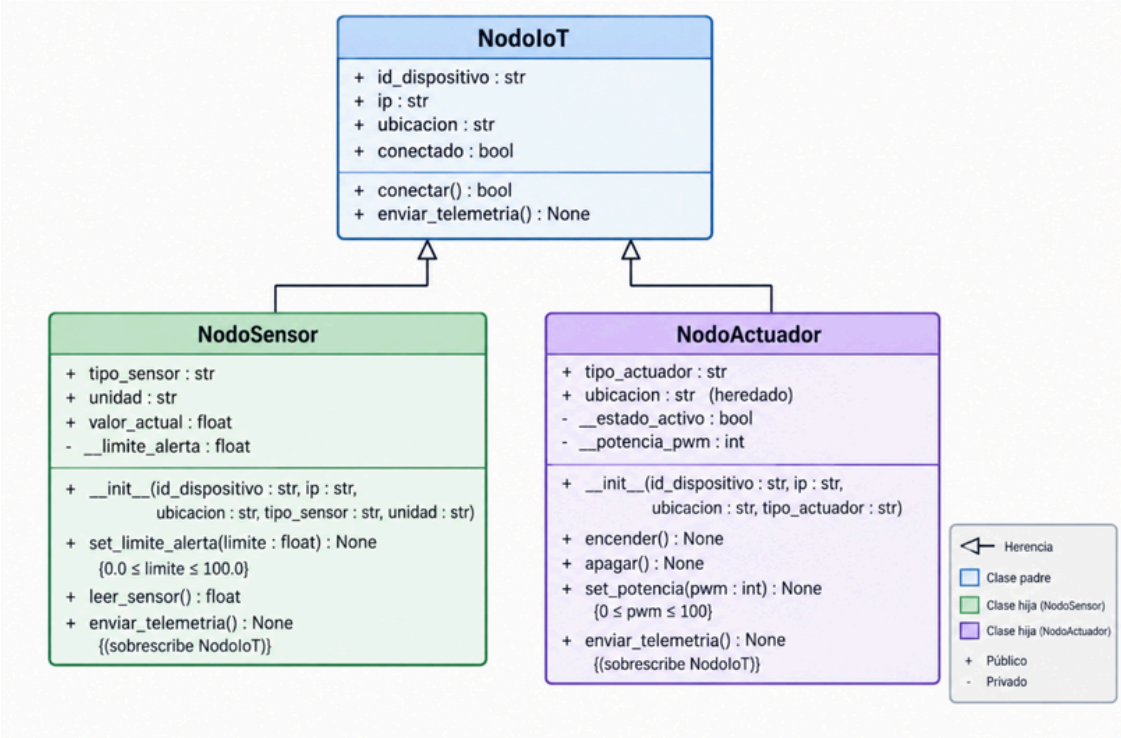
El sistema de control de acceso RFID se diseñó mediante una estructura modular basada en clases. Se consideraron dos dispositivos principales: un lector RFID, encargado de recibir el identificador de las tarjetas, y una cerradura electrónica, responsable de permitir o restringir el acceso.

Para representar ambos dispositivos se utilizó una clase padre llamada NodoIoT, de la cual heredan las clases NodoSensor y NodoActuador.

Adicionalmente, se implementó la clase SistemaAcceso, encargada de comparar los identificadores UID con una lista de tarjetas autorizadas y controlar el estado de la cerradura.

La práctica se desarrolló como una simulación en Python, por lo que no fue necesario utilizar dispositivos físicos para comprobar el funcionamiento de la lógica del programa.

4.2 Diagrama de clases UML



3.3 Implementación de la clase padre

En el archivo nodo_iot.py se desarrolló la clase principal NodoIoT, la cual contiene los atributos compartidos por los dispositivos: identificador, dirección IP, ubicación y estado de conexión.

También se implementaron los métodos conectar() y enviar_telemetria(). El primero permite modificar el estado de conexión del dispositivo, mientras que el segundo muestra su información general.

Esta clase permite reutilizar código en las clases hijas y evita repetir atributos y métodos comunes.

```
1
2   class NodoIoT:
3
4       # Constructor para guardar los datos del dispositivo
5       def __init__(self, id_dispositivo, ip, ubicacion):
6           self.id_dispositivo = id_dispositivo
7           self.ip = ip
8           self.ubicacion = ubicacion
9           self.conectado = False
10          self.bateria = 100
11          self.datos_enviados = 0
12
13      # Funcion para conectar el dispositivo
14      def conectar(self):
15          self.conectado = True
16          print(f"{self.id_dispositivo}: conectado correctamente.")
17
18      def desconectar(self):
19          self.conectado = False
20          print(f"{self.id_dispositivo}: se ha desconectado.")
21
22      # Funcion para enviar los datos del nodo
23      def enviar_telemetria(self):
24          if self.conectado:
25              print("\n--- Datos del dispositivo ---")
26              print(f"ID: {self.id_dispositivo}")
27              print(f"IP: {self.ip}")
28              print(f"Ubicacion: {self.ubicacion}")
29              print(f"Estado: Conectado")
30
31              self.datos_enviados += 1
32              self.bateria -= 1
33
34              print(f"Bateria: {self.bateria}%")
35              print(f"Datos enviados: {self.datos_enviados}")
36          else:
37              print("El dispositivo no esta conectado.")
38
39      # Funcion para consultar el estado del dispositivo
40      def mostrar_estado(self):
41          print("\n--- Estado del nodo ---")
42          print(f"Dispositivo: {self.id_dispositivo}")
43          print(f"Direccion IP: {self.ip}")
44          print(f"Ubicacion: {self.ubicacion}")
45          print(f"Conexion: {self.conectado}")
46          print(f"Bateria: {self.bateria}%")
47          print(f"Total de datos enviados: {self.datos_enviados}")
48
49
50  nodo1 = NodoIoT("ESP32_01", "192.168.1.10", "Laboratorio")
51
52  nodo1.conectar()
53  nodo1.enviar_telemetria()
54  nodo1.enviar_telemetria()
55  nodo1.mostrar_estado()
56  nodo1.desconectar()
```

3.4 Implementación de las clases hijas

En el archivo dispositivos.py se implementaron las clases NodoSensor y NodoActuador, ambas derivadas de NodoIoT.

La clase NodoSensor representa el lector RFID y almacena el identificador UID de la tarjeta detectada. También incluye un atributo privado llamado __limite_alerta y un método setter que valida que el valor asignado sea numérico y no negativo.

Por otra parte, la clase NodoActuador simula una cerradura electrónica mediante los métodos encender() y apagar(). Para proteger su estado interno se utilizaron los atributos privados __estado_activo y __potencia_pwm.

Ambas clases sobrescriben el método enviar_telemetria() para mostrar información específica de cada dispositivo.

```python

    from nodo_iot import NodoIoT
    from numbers import Real


    # Clase hija: lector RFID
    class NodoSensor(NodoIoT):

        def __init__(self, id_dispositivo, ip, ubicacion):
            super().__init__(id_dispositivo, ip, ubicacion)

            self.__limite_alerta = 0.0
            self.lectura_actual = "Sin tarjeta"

        def establecer_limite(self, limite):

            if isinstance(limite, bool) or not isinstance(limite, Real):
                raise ValueError("El limite debe ser numerico.")

            if limite < 0:
                raise ValueError("El limite no puede ser negativo.")

            self.__limite_alerta = float(limite)

        def leer_tarjeta(self, uid):
            self.lectura_actual = uid.strip().upper()
            print(f"Tarjeta detectada: {self.lectura_actual}")

        def enviar_telemetria(self):

            print("\n--- TELEMETRIA DEL LECTOR RFID ---")
            print(f"ID: {self.id_dispositivo}")
            print(f"IP: {self.ip}")
            print(f"Ubicacion: {self.ubicacion}")
            print(f"UID detectado: {self.lectura_actual}")
            print(f"Limite: {self.__limite_alerta}")
            print(f"Conectado: {self.conectado}")


    # Clase hija: cerradura electronica
    class NodoActuador(NodoIoT):
```

```
43      def __init__(self, id_dispositivo, ip, ubicacion):
44          super().__init__(id_dispositivo, ip, ubicacion)
45
46          self.__estado_activo = False
47          self.__potencia_pwm = 0
48
49      def encender(self):
50          self.__estado_activo = True
51          self.__potencia_pwm = 100
52          print("ACCESO CONCEDIDO: Cerradura desbloqueada.")
53
54      def apagar(self):
55          self.__estado_activo = False
56          self.__potencia_pwm = 0
57          print("Cerradura bloqueada.")
58
59      def enviar_telemetria(self):
60
61          estado = "Desbloqueada" if self.__estado_activo else "Bloqueada"
62
63          print("\n--- TELEMETRIA DE LA CERRADURA ---")
64          print(f"ID: {self.id_dispositivo}")
65          print(f"IP: {self.ip}")
66          print(f"Ubicacion: {self.ubicacion}")
67          print(f"Estado: {estado}")
68          print(f"Potencia PWM: {self.__potencia_pwm}%")
69          print(f"Conectado: {self.conectado}")
```

3.5 Implementación del bucle polimórfico

En el archivo main.py se implementó el programa principal, encargado de administrar el funcionamiento del sistema.

Se creó una lista denominada red_iot, que contiene los objetos correspondientes al lector RFID y la cerradura electrónica.

Posteriormente, mediante un ciclo for, se recorrió la lista y se ejecutó el método enviar_telemetria() para cada dispositivo.

Gracias al polimorfismo, cada objeto ejecuta automáticamente la implementación del método correspondiente a su propia clase, sin necesidad de utilizar instrucciones condicionales para determinar el tipo de dispositivo.

También se implementó un menú interactivo que permite introducir identificadores UID, verificar el acceso y consultar la telemetría de la red.

```
98              # Bucle polimorfico
99              for dispositivo in red_iot:
100                 dispositivo.enviar_telemetria()
```

4 Pruebas y resultados

```
IP: 192.168.1.10
Ubicacion: Laboratorio
Estado: Conectado
Bateria: 99%
Datos enviados: 1

--- Datos del dispositivo ---
ID: ESP32_01
IP: 192.168.1.10
Ubicacion: Laboratorio
Estado: Conectado
Bateria: 98%
Datos enviados: 2

--- Estado del nodo ---
Dispositivo: ESP32_01
Direccion IP: 192.168.1.10
Ubicacion: Laboratorio
Conexion: True
Bateria: 98%
Total de datos enviados: 2
ESP32_01: se ha desconectado.
RFID-001: conectado correctamente.
LOCK-001: conectado correctamente.

===== CONTROL DE ACCESO RFID =====

1. Leer tarjeta RFID
2. Mostrar telemetria
3. Salir
Selecciona una opcion: 1
Introduce el UID de la tarjeta: A1B2C3D4
Tarjeta detectada: A1B2C3D4
Tarjeta autorizada.
ACCESO CONCEDIDO: Cerradura desbloqueada.
```

Prueba 1. Acceso Autorizado

```
1. Leer tarjeta RFID
2. Mostrar telemetria
3. Salir
Selecciona una opcion: 1
Introduce el UID de la tarjeta: 99999999
Tarjeta detectada: 99999999
Tarjeta NO autorizada.
Cerradura bloqueada.
```

Prueba 2. Acceso Denegado

```
===== MONITOREO DE RED IoT =====

--- TELEMETRIA DEL LECTOR RFID ---
ID: RFID-001
IP: 192.168.1.10
Ubicacion: Entrada principal
UID detectado: 99999999
Limite: 10.0
Conectado: True

--- TELEMETRIA DE LA CERRADURA ---
ID: LOCK-001
IP: 192.168.1.11
Ubicacion: Puerta principal
Estado: Bloqueada
Potencia PWM: 0%
Conectado: True

1. Leer tarjeta RFID
2. Mostrar telemetria
3. Salir
Selecciona una opcion:
```

Prueba 3. Polimorfismo

5 Conclusión

Durante el desarrollo de esta práctica se logró implementar un sistema modular de control de acceso RFID utilizando los conceptos de programación orientada a objetos en Python.

La herencia permitió reutilizar los atributos y métodos de una clase principal, mientras que el encapsulamiento ayudó a proteger la información interna de los dispositivos y validar determinados valores.

Además, mediante el polimorfismo fue posible ejecutar el mismo método en diferentes objetos, obteniendo resultados específicos para cada dispositivo sin utilizar condicionales para identificar sus clases.

Esta práctica permitió comprender cómo organizar un programa mediante clases y cómo aplicar estos conceptos a un sistema de control de acceso que posteriormente podría integrarse con dispositivos físicos como un lector RFID y una cerradura electrónica.

7. Video demostrativo

Enlace al video de demostración: https://drive.google.com/file/d/1a_VtcNFpXO3DtXL9-1JH2-ksm1kPTbZt/view?usp=sharing